

Python for Data Analytics 2

Session 2: Vectorized String Operations and Regular Expressions

Session Objectives

- Understand why string operations matter in analytics
- Master pandas vectorized string operations with the `str` attribute
- Learn regular expression basics for pattern matching
- Apply regex metacharacters and syntax for business data
- Combine pandas and regex for powerful data cleaning
- Practice with real-world business examples

Why String Operations Matter in Analytics

- **Data cleaning:** Standardizing formats, removing inconsistencies
- **Text preprocessing:** Preparing text data for analysis and modeling
- **Pattern extraction:** Finding emails, phone numbers, codes in unstructured data
- **Data validation:** Checking if data follows expected patterns
- **Business intelligence:** Categorizing and analyzing textual information

The `str` Attribute: Vectorized String Operations

- Apply string methods to entire pandas Series at once
- Automatically handles missing values (None/NaN)
- Much faster than loops for large datasets

```
import pandas as pd
# Customer names from business dataset
data = ["alice johnson", "Bob Smith", None, "MARIA GARCIA"]
customer_names = pd.Series(data)

# Vectorized capitalization – handles None automatically
customer_names.str.capitalize()
# 0 Alice johnson
# 1 Bob smith
# 2 None
# 3 Maria garcia
```

Common Pandas String Methods

Key methods for business data cleaning and analysis:

```
employees = pd.Series(['Sarah Williams', 'Michael Chen', 'Emma Rodriguez'])

employees.str.lower()          # Convert to lowercase
# 0 sarah williams
# 1 michael chen
# 2 emma rodriguez

employees.str.len()           # Get string lengths
# 0 13
# 1 12
# 2 14

employees.str.startswith('S') # Check prefix
# 0 True
# 1 False
# 2 False
```

Vectorized String Indexing and Slicing

Extract parts of strings across entire Series:

```
employees = pd.Series(['Sarah Williams', 'Michael Chen', 'Emma Rodriguez'])

# Extract first 3 characters (useful for employee codes)
employees.str[:3]
# 0 Sar
# 1 Mic
# 2 Emm

# Split names and extract last names
employees.str.split().str[-1]
# 0 Williams
# 1 Chen
# 2 Rodriguez
```

Introduction to Regular Expressions

Regular expressions (regex) are patterns that describe text you want to find or match.

Common problem: inconsistent spacing in business data

```
text = "Product quality is          excellent and delivery was fast"

# Traditional approach fails with multiple spaces
text.split(" ")
# ['Product', 'quality', 'is', '', '', '', '', '', '', '', 'excellent', ...]

# Regex solution: \s+ means "one or more whitespace characters"
import re
regex = re.compile(r'\s+')
regex.split(text)
# ['Product', 'quality', 'is', 'excellent', 'and', 'delivery', 'was', 'fast']
```

What are Raw Strings?

Raw strings prevent Python from interpreting escape sequences like `\n`, `\t`, etc.

```
# Regular strings interpret escape sequences
regular_string = "Hello\nWorld\tTab"
print(regular_string)
# Hello
# World Tab

# Raw strings treat backslashes literally
raw_string = r"Hello\nWorld\tTab"
print(raw_string)
# Hello\nWorld\tTab

# This is crucial for regex patterns that use lots of backslashes
file_path = r"C:\Users\Documents\file.txt" # Raw string for Windows paths
regex_pattern = r"\d+\.\d+" # Raw string for decimal number pattern
```


Raw Strings and Regex Patterns

Always use raw strings (`r''`) for regex patterns to avoid escaping issues:

```
# Raw string prevents Python from interpreting backslashes
pattern = r'\s+' # Matches one or more whitespace characters

print('a\tb')    # a    b (tab interpreted)
print(r'a\tb')    # a\tb (literal backslash-t-b)

# Regex breakdown:
# \s = any whitespace (space, tab, newline)
# + = one or more of the preceding character
```

Essential Regex Metacharacters

Characters with special meanings in regex patterns:

Character	Description	Business Example
.	Any character except newline	prod.ct matches "product", "produce"
^	Start of string	^Order matches "Order" at beginning
\$	End of string	\.com\$ matches ".com" at end
+	One or more repetitions	\d+ matches any number
*	Zero or more repetitions	colou*r matches "color", "colour"
?	Zero or one repetition	colou?r matches "color", "colour"
{n}	Exactly n repetitions	\d{4} matches 4-digit years

Character Classes and Shortcuts

Predefined patterns for common character types:

```
import re

# \d = digits, \w = word characters, \s = whitespace
regex = re.compile(r'\w\s\w') # word char + space + word char
regex.findall('the fox is 9 years old')
# ['e f', 'x i', 's 9', '9 y', 's o', 'd']

# \D = non-digits, \W = non-word chars, \S = non-whitespace
regex = re.compile(r'\W\S') # non-word char + non-whitespace
regex.findall('!a2%e4')
# ['!a', '%e']
```

Custom Character Groups with Square Brackets

Define your own character sets:

```
# Match any vowel
regex = re.compile('[aeiou]')
regex.split('consequential')
# ['c', 'ns', 'q', '', 'nt', '', 'l']

# Match uppercase letter followed by digit (product codes)
regex = re.compile('[A-Z][0-9]')
regex.findall('Product codes: A3, B7, G2, X9')
# ['A3', 'B7', 'G2', 'X9']

# Ranges: [a-z] = any lowercase, [1-3] = digits 1, 2, or 3
```

Repetition Markers

Control how many times a pattern should repeat:

```
# {n} = exactly n times
regex = re.compile(r'\w{3}') # exactly 3 word characters
regex.findall('The quick brown fox')
# ['The', 'qui', 'ck', 'bro', 'wn', 'fox']

# + = one or more times
regex = re.compile(r'\w+') # one or more word characters
regex.findall('The quick brown fox')
# ['The', 'quick', 'brown', 'fox']

# {m,n} = between m and n times
regex = re.compile(r'\w{3,4}') # 3 to 4 word characters
regex.findall('The quick brown fox')
# ['The', 'quic', 'brow', 'fox']
```

Business Example: Email Pattern Matching

Building regex patterns for email validation:

```
# Basic email pattern (simplified)
email_pattern = re.compile(r'\w+@\w+\.[a-z]{3}')

business_text = "Contact sales@company.com or support@help.org"
email_pattern.findall(business_text)
# ['sales@company.com', 'support@help.org']

# Improved pattern for emails with dots in names
improved_email = re.compile(r'[\w.]+@[\w.]+\.[a-z]{2,3}')
improved_email.findall('sarah.johnson@company.com')
# ['sarah.johnson@company.com']
```

Regex Groups: Extracting Components

Use parentheses to capture specific parts of matches:

```
# Extract email components: username, domain, suffix
email_groups = re.compile(r'([\w.]+)@(\w+)\.([a-z]{3})')

text = "Contact guido@python.org for support"
email_groups.findall(text)
# [('guido', 'python', 'org')]

# Named groups for clarity
email_named = re.compile(r'(?P<user>\w+)@(?P<domain>\w+)\. (?P<suffix>[a-z]{3})')
match = email_named.match('guido@python.org')
match.groupdict()
# {'user': 'guido', 'domain': 'python', 'suffix': 'org'}
```

Pandas String Methods with Regex (1)

Setting up our business data for regex operations:

```
import pandas as pd
import re

# Customer data with mixed format - typical business scenario
customers = pd.Series([
    "Sarah Williams - sarah.williams@company.com",
    "Michael Chen - m.chen@business.org",
    "Emma Rodriguez - emma.r@startup.io"
])

print(customers)
# 0      Sarah Williams - sarah.williams@company.com
# 1           Michael Chen - m.chen@business.org
# 2           Emma Rodriguez - emma.r@startup.io
# dtype: object
```

This mixed-format data is common in business: names and emails combined, inconsistent

Pandas String Methods with Regex (2)

Extracting structured information with `str.extract()` :

```
# Extract names (everything before the dash)
# [^–]+ means "one or more characters that are NOT a dash"
names = customers.str.extract(r'([^–]+)')
print(names)
# 0      Sarah Williams
# 1      Michael Chen
# 2      Emma Rodriguez

# The parentheses create a capture group
# str.extract() returns the captured content as a DataFrame
print(type(names))
# <class 'pandas.core.frame.DataFrame'>
```

Perfect for separating mixed data into clean, structured columns!

Pandas String Methods with Regex (3)

Finding multiple matches with `str.findall()` :

```
# Find all email addresses in each row
# [\w.]+ matches word characters and dots
# [a-z]{2,3} matches 2-3 lowercase letters (domain suffix)
emails = customers.str.findall(r'[\w.]+@[ \w.]+\.[a-z]{2,3}')
print(emails)
# 0      [sarah.williams@company.com]
# 1      [m.chen@business.org]
# 2      [emma.r@startup.io]

# Returns a Series of lists – useful when multiple matches possible
# For single emails per row, could also use str.extract()
```

Essential for extracting contact information from unstructured business data!

Essential Pandas Regex Methods

Key methods for business data processing:

Method	Description	Use Case
<code>match()</code>	Check if string starts with pattern	Validate format compliance
<code>extract()</code>	Extract matched groups as columns	Split structured data
<code>findall()</code>	Find all matches in each string	Extract multiple items
<code>replace()</code>	Replace pattern matches	Clean and standardize data
<code>contains()</code>	Check if pattern exists anywhere	Filter and categorize
<code>split()</code>	Split on regex pattern	Parse delimited data

Advanced Pattern Matching Example

Filter companies based on complex criteria:

```
companies = pd.Series(['Amazon Tech', 'Google Inc', 'Microsoft Corp', 'Oracle Systems'])

# Find companies starting with vowel AND ending with consonant
pattern = r'^[AEIOUaeiou].*[bcdfghjklmnpqrstvwxyzBCDFGHJKLMNPQRSTVWXYZ]$\ncompanies.str.match(pattern)\n# 0      True    # Amazon Tech\n# 1     False    # Google Inc\n# 2     False    # Microsoft Corp\n# 3     False    # Oracle Systems
```

Best Practices for Business Analytics

Performance Tips:

- Use pandas vectorized operations instead of loops
- Compile regex patterns once, reuse multiple times
- Test patterns with small data samples first

Development Tips:

- Use raw strings (`r' '`) for all regex patterns
- Start simple, build complexity gradually
- Document complex patterns with comments
- Use online tools like regex101.com for testing

Common Business Use Cases

Data Validation: Phone numbers, postal codes, product IDs

Text Cleaning: Remove extra whitespace, standardize formats

Information Extraction: Extract dates, amounts, codes from text

Data Categorization: Classify text based on patterns

Log Analysis: Parse structured info from unstructured logs

```
# Example: Extract product codes from descriptions
products = pd.Series(['Laptop Model ABC-123', 'Phone XYZ-456', 'Tablet DEF-789'])
products.str.extract(r'([A-Z]{3}-\d{3})')
# 0      ABC-123
# 1      XYZ-456
# 2      DEF-789
```

Summary: String Operations & Regex

- ✓ Master pandas vectorized string operations for efficiency
- ✓ Use the `str` attribute for clean, readable code
- ✓ Apply regex patterns for complex text matching
- ✓ Combine character classes and repetition markers
- ✓ Extract structured information with regex groups
- ✓ Integrate pandas and regex for powerful data cleaning
- ✓ Follow best practices for maintainable code

Mastering these tools transforms messy text data into valuable business insights.